Chapter 3
Project 1.1: Data Acquisition Base Application

The beginning of the data pipeline is acquiring the raw data from various sources. This chapter has a single project to create a **command-line application** (**CLI**) that extracts relevant data from files in CSV format. This initial application will restructure the raw data into a more useful form. Later projects (starting in ***Chapter 9***, ***Project 3.1: Data Cleaning Base Application***) will add features for cleaning and validating the data.

This chapter's project covers the following essential skills:

- Application design in general. This includes an object-oriented design and the SOLID design principles, as well as functional design.
- A few CSV file processing techniques. This is a large subject area, and the project focuses on restructuring source data into a more usable form.
- CLI application construction.
- Creating acceptance tests using the Gherkin language and **behave** step definitions.
- Creating unit tests with mock objects.

We'll start with a description of the application, and then move on to talk about the architecture and construction. This will be followed by a detailed list of deliverables.

3.1 Description

Analysts and decision-makers need to acquire data for further analysis. In many cases, the data is available in CSV-formatted files. These files may be extracts from databases or downloads from web services.

For testing purposes, it's helpful to start with something relatively small. Some of the Kaggle data sets are very, very large, and require sophisticated application design. One of the most fun small data sets to work with is Anscombe's Quartet. This can serve as a test case to understand the issues and concerns in acquiring raw data.

We're interested in a few key features of an application to acquire data:

- When gathering data from multiple sources, it's imperative to convert it to a common format. Data sources vary, and will often change with software upgrades. The acquisition process needs to be flexible with respect to data sources and avoid assumptions about formats.
- A CLI application permits a variety of automation possibilities. For example, a CLI application can be "wrapped" to create a web service. It can be run from the command line manually, and it can be automated through enterprise job scheduling applications.
- The application must be extensible to reflect source changes. In many cases, enterprise changes are not communicated widely enough, and data analysis applications discover changes "the hard way" — a source of data suddenly includes unexpected or seemingly invalid values.

3.1.1 User experience

The **User Experience** (**UX**) will be a command-line application with options to fine-tune the data being gathered. This essential UX pattern will be used for many of this book's projects. It's flexible and can be made to run almost anywhere.

Our expected command line should look something like the following:

```
% python src/acquire.py -o quartet Anscombe_quartet_data.csv
```

The `-o quartet` argument specifies a directory into which the resulting extracts are written. The source file contains four separate series of data. Each of the series can be given an unimaginative name like `quartet/series_1.json`.

The positional argument, `Anscombe_quartet_data.csv`, is the name of the downloaded source file.

While there's only one file – at the present time – a good design will work with multiple input files and multiple source file formats.

In some cases, a more sophisticated "dashboard" or "control panel" application might be desirable as a way to oversee the operation of the data acquisition process. The use of a web-based API can provide a very rich interactive experience. An alternative is to use tools like **rich** or **Textual** to build a small text-based display. Either of these choices should be built as a wrapper that executes the essential CLI application as a subprocess.

Now that we've seen an overview of the application's purpose and UX, let's take a look at the source data.

3.1.2 About the source data

Here's the link to the dataset we'll be using:

https://www.kaggle.com/datasets/carlmcbrideellis/data-anscombes-quartet

You'll need to register with Kaggle to download this data.

The Kaggle URL presents a page with information about the CSV-formatted file. Clicking the **Download** button will download the small file of data to your local computer.

The data is available in this book's GitHub repository's `data` folder, also.

Once the data is downloaded, you can open the `Anscombe_quartet_data.csv` file to inspect the raw data.

The file contains four series of (x,y) pairs in each row. We can imagine each row as having $[(x_1,y_1),(x_2,y_2),(x_3,y_3),(x_4,y_4)]$. It is, however, compressed, as we'll see below.

We might depict the idea behind this data with an entity-relationship diagram as shown in ***Figure 3.1***.

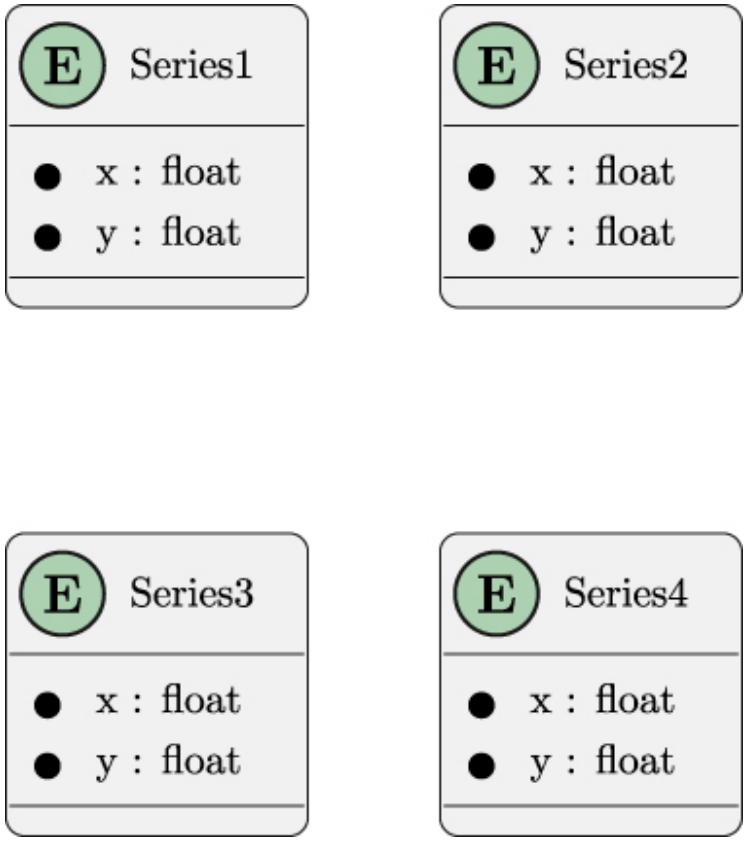

Figure 3.1: Notional entity-relationship diagram

Interestingly, the data is not organized as four separate (x,y) pairs. The downloaded file is organized as follows:

$$[x_{1,2,3}, y_1, y_2, y_3, x_4, y_4]$$

We can depict the actual source entity type in an ERD, as shown in ***Figure 3.2***.

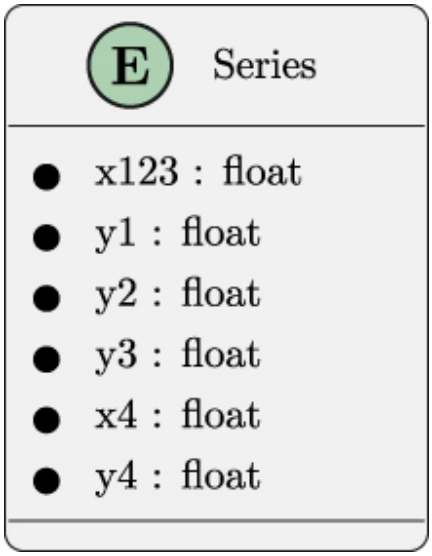

Figure 3.2: Source entity-relationship diagram

One part of this application's purpose is to disentangle the four series into separate files. This forces us to write some transformational processing to rearrange each row's data elements into four separate data sets.

The separate series can then be saved into four separate files. We'll look more deeply at the details of creating the separate files for a separate project in ***Chapter 11***, ***Project 3.7: Interim Data Persistence***. For this project, any file format for the four output files will do nicely; ND JSON serialization is often ideal.

We encourage you to take a look at the file before moving on to consider how it needs to be transformed into distinct output files.

Given this compressed file of source data, the next section will look at the expanded output files. These will separate each series to make them easier to work with.

3.1.3 About the output data

The ND JSON file format is described in **http://ndjson.org** and **https://jsonlines.org**. The idea is to put each individual entity into a JSON document written as a single physical line. This fits with the way the Python `json.dumps()` function works: if no value is provided for the

`indent` parameter (or if the value is `indent=None`), the text will be as compact as possible.

The `series_1.json` output file should start like this:

```
{"x": "10.0", "y": "8.04"}
{"x": "8.0", "y": "6.95"}
{"x": "13.0", "y": "7.58"}
...
```

Each row is a distinct, small JSON document. The row is built from a subset of fields in the input file. The values are strings: we won't be attempting any conversions until the cleaning and validating projects in ***Chapter 9***, ***Project 3.1: Data Cleaning Base Application***.

We'll require the user who runs this application to create a directory for the output and provide the name of the directory on the command line. This means the application needs to present useful error messages if the directory doesn't actually exist. The `pathlib.Path` class is very helpful for confirming a directory exists.

Further, the application should be cautious about overwriting any existing files. The `pathlib.Path` class is very helpful for confirming a file already exists.

This section has looked at the input, processing, and output of this application. In the next section, we'll look at the overall architecture of the software.

3.2 Architectural approach

We'll take some guidance from the C4 model (**https://c4model.com**) when looking at our approach.

- **Context**: For this project, a context diagram would show a user extracting data from a source. You may find it helpful to draw this diagram.

- **Containers**: This project will run on the user's personal computer. As with the context, the diagram is small, but some readers may find it helpful to take the time to draw it.
- **Components**: We'll address these below.
- **Code**: We'll touch on this to provide some suggested directions.

We can decompose the software architecture into these two important components:

- `model`: This module has definitions of target objects. In this project, there's only a single class here.
- `extract`: This module will read the source document and creates model objects.

Additionally, there will need to be these additional functions:

- A function for parsing the command-line options.
- A `main()` function to parse options and do the file processing.

As suggested in ***Chapter 1***, ***Project Zero: A Template for Other Projects***, the initialization of logging will often look like the following example:

```
if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    main()
```

The idea is to write the `main()` function in a way that maximizes reuse. Avoiding logging initialization means other applications can more easily import this application's `main()` function to reuse the data acquisition features.

Initializing logging within the `main()` function can undo previous logging initialization. While there are ways to have a composite application tolerate each `main()` function doing yet another initialization of logging, it seems simpler to refactor this functionality outside the important processing.

For this project, we'll look at two general design approaches for the model and extract components. We'll utilize this opportunity to highlight the importance of adhering to SOLID design principles.

First, we'll show an object-oriented design using class definitions. After that, we'll show a functional design, using only functions and stateless objects.

3.2.1 Class design

One possible structure for the classes and functions of this application is shown in ***Figure 3.3***.

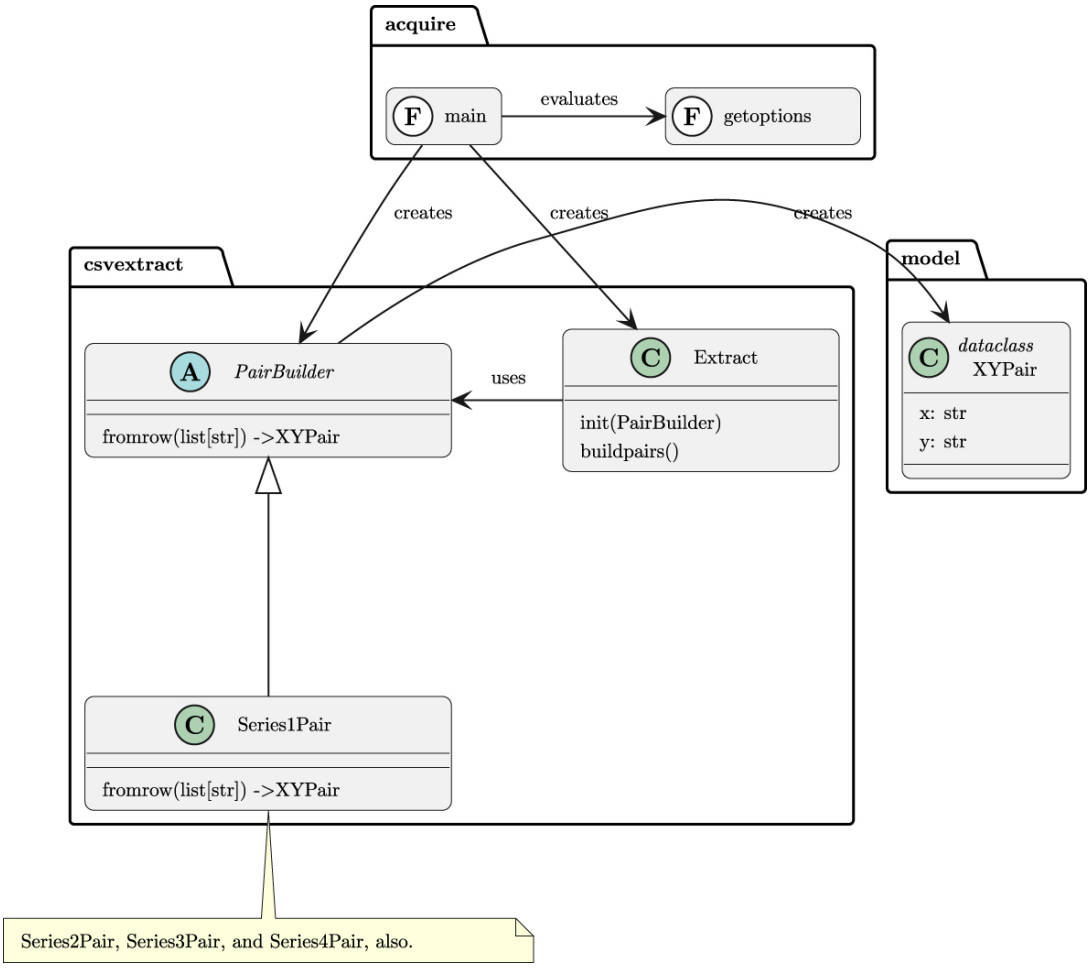

Figure 3.3: Acquisition Application Model

The `model` module contains a single class definition for the raw `XYPair`. Later, this is likely to expand and change. For now, it can seem like over-engineering.

The `acquisition` module contains a number of classes that collaborate to build `XYPair` objects for any of the four series. The abstract `PairBuilder` class defines the general features of creating an `XYPair` object.

Each subclass of the `PairBuilder` class has a slightly different implementation. Specifically, the `Series1Pair` class has a `from_row()` method that assembles a pair from the $x_{1,2,3}$ and y_1 values. Not shown in the diagram are the three other subclasses that use distinct pairs of columns to create `XYPair` objects from the four series.

The diagram and most of the examples here use `list[str]` as the type for a row from a CSV reader.

If a `csv.DictReader` is used, the source changes from `list[str]` to

`dict[str, str]`. This small, but important, change will ripple throughout the examples.

In many cases, it seems like a `csv.DictReader` is a better choice. Column names can be provided if the CSV file does not have names in the first row.

We've left the revisions needed for this change as part of the design work for you.

The overall `Extract` class embodies the various algorithms for using an instance of the `PairBuilder` class and a row of source data to build `XYPair` instances. The `build_pair(list[str]) -> XYPair` method makes a single item from a row parsed from a CSV file.

The job of the `main()` function is to create instances of each of the four `PairBuilder` subclasses. These instances are then used to create four instances of the `Extract` class. These four `Extract` objects can then build four `XYPair` objects from each source row.

The `dataclass.asdict()` function can be used to convert an `XYPair` object into a `dict[str, str]` object. This can be serialized by `json.dumps()` and written to an appropriate output file. This conversion operation seems like a good choice for a method in the abstract `PairBuilder` class. This can be used to write an `XYPair` object to an open file.

The top-level functions, `main()` and `get_options()`, can be placed in a separate module, named `acquisition`. This module will import the various class definitions from the `model` and `csv_extract` modules.

It's often helpful to review the SOLID design principles. In particular, we'll look closely at the **Dependency Inversion principle**.

3.2.2 Design principles

We can look at the SOLID design principles to be sure that the object-oriented design follows those principles.

- **Single Responsibility**: Each of the classes seems to have a single responsibility.
- **Open-Closed**: Each class seems open to extension by adding subclasses.
- **Liskov Substitution**: The `PairBuilder` class hierarchy follows this principle since each subclass is identical to the parent class.
- **Interface Segregation**: The interfaces for each class are minimized.
- **Dependency Inversion**: There's a subtle issue regarding dependencies among classes. We'll look at this in some detail.

One of the SOLID design principles suggests avoiding tight coupling between the `PairBuilder` subclasses and the `XYPair` class. The idea would be to provide a protocol (or interface) for the `XYPair` class. Using the protocol in type annotations would permit any type that implemented the protocol to be provided to the class. Using a protocol would break a direct dependency between the `PairBuilder` subclasses and the `XYPair` class.

This object-oriented design issue surfaces often, and generally leads to drawn-out, careful thinking about the relationships among classes and the SOLID design principles.

We have the following choices:

- Have a direct reference to the `XYPair` class inside the `PairBuilder` class. This would be `def from_row(row: list[str]) -> XYPair:`. This breaks the Dependency Inversion principle.
- Use `Any` as the type annotation. This would be `def from_row(row: list[str]) -> Any:`. This makes the type hints less informative.
- Attempt to create a protocol for the resulting type, and use this in the type hints.
- Introduce a type alias that (for now) only has one value. In future expansions of the `model` module, additional types might be introduced.

The fourth alternative gives us the flexibility we need for type annotation checking. The idea is to include a type alias like the following in the `model` module:

```
from dataclasses import dataclass
from typing import TypeAlias

@dataclass
class XYPair:
    # Definition goes here

RawData: TypeAlias = XYPair
```

As alternative classes are introduced, the definition of `RawData` can be expanded to include the alternatives. This might evolve to look like the following:

```
from dataclasses import dataclass
from typing import TypeAlias

@dataclass
```

```
class XYPair:
    # Definition goes here
    pass

@dataclass
class SomeOtherStructure:
    # Some other definition, here
    pass

RawData: TypeAlias = XYPair | SomeOtherStructure
```

This permits extension to the `PairBuilder` subclasses as the `model` module evolves. The `RawData` definition needs to be changed as new classes are introduced. Annotation-checking tools like **mypy** cannot spot the invalid use of any of the classes that comprise the alternative definitions of the `RawData` type alias.

Throughout the rest of the application, classes and functions can use `RawData` as an abstract class definition. This name represents a number of alternative definitions, any one of which might be used at run-time.

With this definition of `RawData`, the `PairBuilder` subclasses can use a definition of the following form:

```
from model import RawData, XYPair
from abc import ABC, abstractmethod

class PairBuilder(ABC):
    target_class: type[RawData]

    @abstractmethod
    def from_row(self, row: list[str]) -> RawData:
        ...

class Series1Pair(PairBuilder):
    target_class = XYPair

    def from_row(self, row: list[str]) -> RawData:
        cls = self.target_class
```

```
            # the rest of the implementation...
            # return cls(arguments based on the value of row)
```

A similar analysis holds for the `main()` function. This can be directly tied to the `Extract` class and the various subclasses of the `PairBuilder` class. It's very important for these classes to be injected at run time, generally based on command-line arguments.

For now, it's easiest to provide the class names as default values. A function like the following might be used to get options and configuration parameters:

```
def get_options(argv: list[str]) -> argparse.Namespace:
    defaults = argparse.Namespace(
        extract_class=Extract,
        series_classes=[Series1Pair, Series2Pair, Series3Pair, Series4Pair],
    )
```

The `defaults` namespace is provided as an argument value to the `ArgumentParser.parse_args()` method. This set of defaults serves as a kind of dependency injection throughout the application. The `main` function can use these class names to build an instance of the given extract class, and then process the given source files.

A more advanced CLI could provide options and arguments to tailor the class names. For more complex applications, these class names would be read from a configuration file.

An alternative to the object-oriented design is a functional design. We'll look at that alternative in the next section.

3.2.3 Functional design

The general module structure shown in ***Class design*** applies to a functional design also. The `model` module with a single class definition is also a part of a functional design; this kind of module with a collection of dataclass definitions is often ideal.

As noted above in the ***Design principles*** section, the `model` module is best served by using a type variable, `RawData`, as a placeholder for any additional types that may be developed.

The `csv_extract` module will use a collection of independent functions to build `XYPair` objects. Each function will be similar in design.

Here are some example functions with type annotations:

```
def series_1_pair(row: list[str]) -> RawData:
    ...

def series_2_pair(row: list[str]) -> RawData:
    ...

def series_3_pair(row: list[str]) -> RawData:
    ...

def series_4_pair(row: list[str]) -> RawData:
    ...
```

These functions can then be used by an `extract()` function to create the `XYPair` objects for each of the four series represented by a single row of the source file.

One possibility is to use the following kind of definition:

```
SeriesBuilder: TypeVar = Callable[[list[str]], RawData]

def extract(row: list[str], builders: list[SeriesBuilder]) -> list[RawData]:
    ...
```

This `extract()` function can then apply all of the given builder functions (`series_1_pair()` to `series_4_pair()`) to the given row to create `XYPair` objects for each of the series.

This design will also require a function to apply `dataclass.asdict()` and `json.dumps()` to convert `XYPair` objects into strings that can be written to an NDJSON file.

Because the functions used are provided as argument values, there is little possibility of a dependency issue among the various functions that make up the application. The point throughout the design is to avoid binding specific functions in arbitrary places. The `main()` function should provide the row-building functions to the `extract` function. These functions can be provided via command-line arguments, a configuration file, or be default values if no overrides are given.

We've looked at the overall objective of the project, and two suggested architectural approaches. We can now turn to the concrete list of deliverables.

3.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for model module classes in the `tests` folder.
- Mock objects for the `csv_extract` module tests will be part of the unit tests.
- Unit tests for the `csv_extract` module components in the `tests` folder.
- Application to acquire data from a CSV file in the `src` folder.

An easy way to start is by cloning the project zero directory to start this project. Be sure to update the `pyproject.toml` and `README.md` when cloning; the author has often been confused by out-of-date copies of old projects' metadata.

We'll look at a few of these deliverables in a little more detail. We'll start with some suggestions for creating the acceptance tests.

3.3.1 Acceptance tests

The acceptance tests need to describe the overall application's behavior from the user's point of view. The scenarios will follow the UX concept of a command-line application that acquires data and writes output files. This includes success as well as useful output in the event of failure.

The features will look something like the following:

```
Feature: Extract four data series from a file with
the peculiar Anscombe Quartet format.

Scenario: When requested, the application extracts all four series.
  Given the "Anscombe_quartet_data.csv" source file exists
  And the "quartet" directory exists
  When we run
    command "python src/acquire.py -o quartet Anscombe_quartet_data.csv"
  Then the "quartet/series_1.json" file exists
  And the "quartet/series_2.json" file exists
  And the "quartet/series_3.json" file exists
  And the "quartet/series_3.json" file exists
  And the "quartet/series_1.json" file starts with
    '{"x": "10.0", "y": "8.04"}'
```

This more complex feature will require several step definitions. These include the following:

- `@given('The "{name}" source file exists')`. This function should copy the example file from a source data directory to the temporary directory used to run the test.
- `@given('the "{name}" directory exists')`. This function can create the named directory under the directory used to run the test.
- `@then('the "{name}" file exists')`. This function can check for the presence of the named file in the output directory.
- `@then('the "quartet/series_1.json" file starts with ...')`. This function will examine the first line of the output file. In the event the test fails, it will be helpful to display the contents of the file to help debug the problem. A simple `assert` statement might not be ideal; a more elaborate `if` statement is needed to write debugging output and raise an `AssertionError` exception.

Because the application under test consumes and produces files, it is best to make use of the **behave** tool's `environment.py` module to define two functions to create (and destroy) a temporary directory used when running the test. The following two functions are used by **behave** to do this:

- `before\_scenario(context, scenario)`: This function can create a directory. The `tempfile` module has a `mkdtemp()` function that handles this. The directory needs to be placed into the context so it can be removed.
- `after_scenario(context, scenario)`: This function can remove the temporary directory.

The format for one of the `Then` clauses has a tiny internal inconsistency. The following uses a mixture of `"` and `'` to make it clear where values are inserted into the text:

```
  And the "quartet/series_1.json" file starts with'{"x": "10.0", "y": "8.04"}'
```

Some people may be bothered by the inconsistency. One choice is to use `'` consistently. When there aren't too many feature files, this pervasive change is easy to make. Throughout the book, we'll be inconsistent, leaving the decision to make changes for consistency up to you.

Also, note the `When` clause command is rather long and complicated. The general advice when writing tests like this is to use a summary of the command and push the details into the step implementation function. We'll address this in a later chapter when the command becomes even longer and more complicated.

In addition to the scenario where the application works correctly, we also need to consider how the application behaves when there are problems. In the next section, we'll touch on the various ways things might go badly, and how the application should behave.

3.3.2 Additional acceptance scenarios

The suggested acceptance test covers only one scenario. This single scenario — where everything works — can be called the ”happy path”. It would be wise to include scenarios in which various kinds of errors occur, to be sure the application is reliable and robust in the face of problems. Here are some suggested error scenarios:

- Given the `Anscombe\_quartet\_data.csv` source file does not exist.
- Given the `quartet` directory does not exist.
- When we run the command `python src/acquire.py --unknown option`
- Given an `Anscombe\_quartet\_data.csv` source file exists, and the file is in the wrong format. There are numerous kinds of formatting problems.
  - The file is empty.
  - The file is not a proper CSV file, but is in some other format.
  - The file’s contents are in valid CSV format, but the column names do not match the expected column names.

Each of the unhappy paths will require examining the log file to be sure it has the expected error messages. The **behave** tool can capture logging information. The `context` available in each step function has attributes that include captured logging output. Specifically, `context.log_capture` contains a `LogCapture` object that can be searched for an error message.

See **https://behave.readthedocs.io/en/stable/api.html#behave.runner.Context** for the content of the context.

These unhappy path scenarios will be similar to the following:

```
Scenario: When the file does not exist, the log has the expected
error message.
  Given the "Anscombe_quartet_data.csv" source file does not exist
  And the "quartet" directory exists
  When we run command "python src/acquire.py -o quartet
```

```
    Anscombe_quartet_data.csv"
    Then the log contains "File not found: Anscombe_quartet_data.csv"
```

This will also require some new step definitions to handle the new `Given` and `Then` steps.

When working with Gherkin, it's helpful to establish clear language and consistent terminology. This can permit a few step definitions to work for a large number of scenarios. It's a common experience to recognize similarities after writing several scenarios, and then choose to alter scenarios to simplify and normalize steps.

The **behave** tool will extract missing function definitions. The code snippets can be copied and pasted into a steps module.

Acceptance tests cover the application's overall behavior. We also need to test the individual components as separate units of code. In the next section, we'll look at unit tests and the mock objects required for those tests.

3.3.3 Unit tests

There are two suggested application architectures in ***Architectural approach***. Class-based design includes two functions and a number of classes. Each of these classes and functions should be tested in isolation.

Functional design includes a number of functions. These need to be tested in isolation. Some developers find it easier to isolate function definitions for unit testing. This often happens because class definitions may have explicit dependencies that are hard to break.

We'll look at a number of the test modules in detail. We'll start with tests for the `model` module.

Unit testing the model

The `model` module only has one class, and that class doesn't really do very much. This makes it relatively easy to test. A test function some-

thing like the following should be adequate:

```
from unittest.mock import sentinel
from dataclasses import asdict

def test_xypair():
    pair = XYPair(x=sentinel.X, y=sentinel.Y)
    assert pair.x == sentinel.X
    assert pair.y == sentinel.Y
    assert asdict(pair) == {"x": sentinel.X, "y": sentinel.Y}
```

This test uses the `sentinel` object from the `unittest.mock` module. Each `sentinel` attribute — for example, `sentinel.X` — is a unique object. They're easy to provide as argument values and easy to spot in results.

In addition to testing the `model` module, we also need to test the `csv_extract` module, and the overall `acquire` application. In the next section, we'll look at the extract unit test cases.

Unit testing the PairBuilder class hierarchy

When following an object-oriented design, the suggested approach is to create a `PairBuilder` class hierarchy. Each subclass will perform slightly different operations to build an instance of the `XYPair` class.

Ideally, the implementation of the `PairBuilder` subclasses is not tightly coupled to the `XYPair` class. There is some advice in the ***Design principles*** section on how to support dependency injection via type annotations. Specifically, the `model` module is best served by using a type variable, `RawData`, as a placeholder for any additional types that may be developed.

When testing, we want to replace this class with a mock class to assure that the interface for the family of `RawData` classes — currently only a single class, `XYPair` — is honored.

A `Mock` object (built with the `unittest.mock` module) works out well as a replacement class. It can be used for the `XYPair` class in the subclasses of the `PairBuilder` class.

The tests will look like the following example:

```
from unittest.mock import Mock, sentinel, call

def test_series1pair():
    mock_raw_class = Mock()
    p1 = Series1Pair()
    p1.target_class = mock_raw_class
    xypair = p1.from_row([sentinel.X, sentinel.Y])
    assert mock_raw_class.mock_calls == [
        call(sentinel.X, sentinel.Y)
    ]
```

The idea is to use a `Mock` object to replace the specific class defined in the `Series1Pair` class. After the `from_row()` method is evaluated, the test case confirms that the mock class was called exactly once with the expected two `sentinel` objects. A further check would confirm that the value of `xypair` was also a mock object.

This use of `Mock` objects guarantees that no additional, unexpected processing was done on the objects. The interface for creating a new `XYPair` was performed correctly by the `Series1Pair` class.

Similar tests are required for the other pair-building classes.

In addition to testing the `model` and `csv_extract` modules, we also need to test the overall `acquire` application. In the next section, we'll look at the `acquire` application unit test cases.

Unit testing the remaining components

The test cases for the overall `Extract` class will also need to use `Mock` objects to replace components like a `csv.reader` and instances of the `PairBuilder` subclasses.

As noted above in the ***Functional design*** section, the `main()` function needs to avoid having explicitly named classes or functions. The names need to be provided via command-line arguments, a configuration file, or as default values.

The unit tests should exercise the `main()` function with `Mock` objects to be sure that it is defined with flexibility and extensions in mind.

3.4 Summary

This chapter introduced the first project, the Data Acquisition Base Application. This application extracts data from a CSV file with a complex structure, creating four separate series of data points from a single file.

To make the application complete, we included a command-line interface and logging. This will make sure the application behaves well in a controlled production environment.

An important part of the process is designing an application that can be extended to handle data from a variety of sources and in a variety of formats. The base application contains modules with very small implementations that serve as a foundation for making subsequent extensions.

Perhaps the most difficult part of this project is creating a suite of acceptance tests to describe the proper behavior. It's common for developers to compare the volume of test code with the application code and claim testing is taking up "too much" of their time.

Pragmatically, a program without automated tests cannot be trusted. The tests are every bit as important as the code they're exercising.

The unit tests are — superficially — simpler. The use of mock objects makes sure each class is tested in isolation.

This base application acts as a foundation for the next few chapters. The next chapter will add RESTful API requests. After that, we'll have database access to this foundation.

3.5 Extras

Here are some ideas for you to add to this project.

3.5.1 Logging enhancements

We skimmed over logging, suggesting only that it's important and that the initialization for logging should be kept separate from the processing within the `main()` function.

The `logging` module has a great deal of sophistication, however, and it can help to explore this. We'll start with logging "levels".

Many of our logging messages will be created with the `INFO` level of logging. For example:

```
  logger.info("%d rows processed", input_count)
```

This application has a number of possible error situations that are best reflected with **error**-level logging.

Additionally, there is a tree of named loggers. The root logger, named `""`, has settings that apply to all the lower-level loggers. This tree tends to parallel the way object inheritance is often used to create classes and subclasses. This can make it advantageous to create loggers for each class. This permits setting the logging level to **debug** for one of many classes, allowing for more focused messages.

This is often handled through a logging configuration file. This file provides the configuration for logging, and avoids the potential complications of setting logging features through command-line options.

There are three extras to add to this project:

- Create loggers for each individual class.
- Add debug-level information. For example, the `from_row()` function is a place where debugging might be helpful for understanding why an output file is incorrect.
- Get the logging configuration from an initialization file. Consider using a file in **TOML** format as an alternative to the **INI** format, which is a first-class part of the `logging` module.

3.5.2 Configuration extensions

We've described a little of the CLI for this application. This chapter has provided a few examples of the expected behavior. In addition to command-line parameters, it can help to have a configuration file that provides the slowly changing details of how the application works.

In the discussion in the ***Design principles*** section, we looked closely at dependency inversion. The intent is to avoid an explicit dependency among classes. We want to "invert" the relationship, making it indirect. The idea is to inject the class name at run time, via parameters.

Initially, we can do something like the following:

```
EXTRACT_CLASS: type[Extract] = Extract
BUILDER_CLASSES: list[type[PairBuilder]] = [
    Series1Pair, Series2Pair, Series3Pair, Series4Pair]

def main(argv: list[str]) -> None:
    builders = [cls() for vls in BUILDER_CLASSES]
    extractor = EXTRACT_CLASS(builders)
    # etc.
```

This provides a base level of parameterization. Some global variables are used to "inject" the run-time classes. These initializations can be moved to the `argparse.Namespace` initialization value for the `ArgumentParser.parse_args()` method.

The initial values for this `argparse.Namespace` object can be literal values, essentially the same as shown in the global variable parameterization shown in the previous example.

It is more flexible to have the initial values come from a parameter file that's separate from the application code. This permits changing the configuration without touching the application and introducing bugs through inadvertent typing mistakes.

There are two popular alternatives for a configuration file that can be used to fine-tune the application. These are:

- A separate Python module that's imported by the application. A module name like `config.py` is popular for this.
- A non-Python text file that's read by the application. The TOML file format, parsed by the `tomllib` module, is ideal.

Starting with Python 3.11, the `tomllib` module is directly available as part of the standard library. Older versions of Python should be upgraded to 3.11 or later.

When working with a TOML file, the class name will be a string. The simple and reliable way to translate the class name from string to class object is to use the `eval()` function. An alternative is to provide a small dictionary with class name strings and class objects. Class names can be resolved through this mapping.

Some developers worry that the `eval()` function allows a class of Evil Super Geniuses to tweak the configuration file in a way that will crash the application.

What these developers fail to notice is that the entire Python application is plain text. The Evil Super Genius can more easily edit the application and doesn't need to do complicated, nefarious things to the parameter file.

Further, ordinary OS-level ownership and permissions can restrict access to the parameter file to a few trustworthy individuals.
Don't forget to include unit test cases for parsing the parameter file. Also, an acceptance test case with an invalid parameter file will be an important part of this project.

3.5.3 Data subsets

To work with large files it will be necessary to extract a subset of the data. This involves adding features like the following:

- Create a subclass of the `Extract` class that has an upper limit on the number of rows created. This involves a number of unit tests.
- Update the CLI options to include an optional upper limit. This, too, will involve some additional unit test cases.
- Update the acceptance test cases to show operation with the upper limit.

Note that switching from the `Extract` class to the `SubsetExtract` class is something that should be based on an optional command-line parameter. If the `--limit` option is not given, then the `Extract` class is used. If the `--limit` option is given (and is a valid integer), then the `SubsetExtract` class is used. This will lead to an interesting set of unit test cases to make sure the command-line parsing works properly.

3.5.4 Another example data source

Perhaps the most important extra for this application is to locate another data source that's of interest to you.

See the **CO2 PPM — Trends in Atmospheric Carbon Dioxide** data set, available at **https://datahub.io/core/co2-ppm**, for some data that's somewhat larger. This has a number of odd special-case values that we'll explore in ***Chapter 6***, ***Project 2.1: Data Inspection Notebook***.

This project will require you to manually download and unzip the file. In later chapters, we'll look at automating these two steps. See ***Chapter 4***, ***Data Acquisition Features: Web APIs and Scraping*** specifically, for projects that will expand on this base project to properly acquire the raw data from a CSV file.

What's important is locating a source of data that's in CSV format and small enough that it can be processed in a few seconds. For large files, it will be necessary to extract a subset of the data. See ***Data subsets*** for advice on handling large sets of data.